Iteration & Refinement

The Prompt Engineering Workflow

Day 2 - Session 4

KavinSchool

Session 4: What We'll Cover

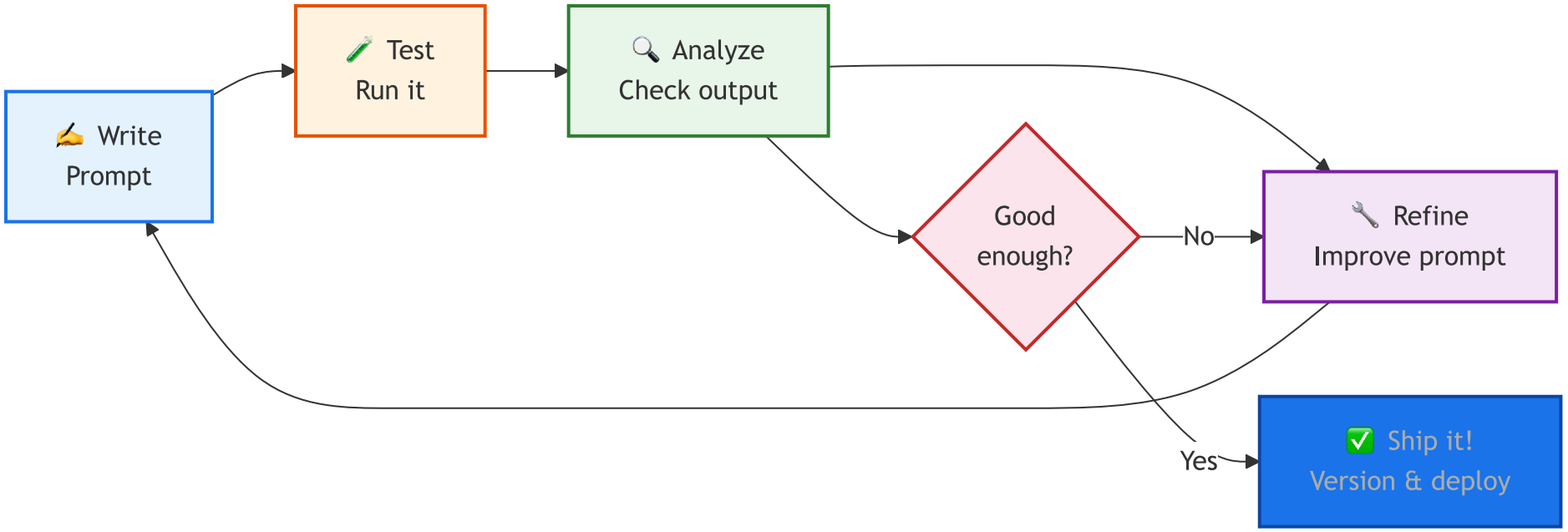
1. The prompt iteration loop
2. Evaluating outputs: manual rubrics & golden datasets
3. LLM-as-judge (introduction)
4. Versioning prompts in code
5. **Lab 2.5:** Refactor Assistant, iterate 3 times
6. Day 2 recap & Day 3 preview

The Prompt Iteration Loop

Write → Test → Analyze → Refine → Repeat

KavinSchool

The Prompt Iteration Loop



Iteration in Practice

Step	What You Do	Time
Write	Draft the prompt with all 4 building blocks	5 min
Test	Run it against 5-10 diverse inputs	5 min
Analyze	Read every output carefully. Where does it fail?	10 min
Refine	Fix the specific failure mode	5 min
Repeat	Test again. New failures? Fix those too	5 min

> A prompt is never "done" on the first try. Plan for **3-5 iterations** minimum.

Common Failure Modes and Fixes

Failure	Symptom	Fix
Wrong format	Markdown instead of JSON	Add stricter format instructions + example
Too verbose	500 words when you wanted 50	Add word/sentence limit in prompt
Missing fields	JSON missing required keys	Provide a complete example in the prompt
Hallucination	Invents facts not in the input	Add "Only use information from the provided text"
Inconsistent	Different format each run	Lower temperature, add few-shot examples
Off-topic	Answers a different question	Make the instruction more specific

Iteration Example: Code Review Prompt

v1: "Review this code"

→ Generic, unfocused output

v2: "Review this Python function for bugs, security issues, and performance problems"

→ Better focus, but output format varies

v3: + "Respond as a numbered list with severity (HIGH/MEDIUM/LOW)"

→ Consistent format, but misses edge cases

v4: + Added 2 few-shot examples showing expected output

→ Much better! But still verbose

v5: + "Maximum 5 findings. One sentence per finding."

→ Concise, consistent, actionable. Ship it!

Evaluating Outputs

How Do You Know If Your Prompt Is Good?

KavinSchool

Evaluation Strategy 1: Manual Rubrics

Create a **scoring rubric** and grade outputs by hand:

```
RUBRIC = {
    "accuracy":    "Is the output factually correct? (1–5)",
    "format":      "Does it match the requested format? (1–5)",
    "completeness":"Does it cover all required fields? (1–5)",
    "conciseness": "Is it the right length? (1–5)",
    "actionable":  "Can the user act on this output? (1–5)",
}
```

Evaluation Strategy 1: Manual Rubrics

Input	accuracy	format	completeness	conciseness	actionable	Total
Test 1	5	5	4	5	5	**24/25**
Test 2	4	5	3	5	4	**21/25**
Test 3	5	3	5	2	5	**20/25**

> Manual rubrics are tedious but essential for your first 10-20 test cases.

Evaluation Strategy 2: Golden Datasets

A **golden dataset** is a set of inputs with **expected outputs** that you've manually verified:

```
GOLDEN_DATASET = [
    {
        "input": "The app crashes when I click save",
        "expected_category": "bug_report",
        "expected_priority": "high",
    },
    {
        "input": "Can you add dark mode?",
        "expected_category": "feature_request",
        "expected_priority": "low",
    },
    # ... 20-50 examples
]
```

Evaluation Strategy 2: Golden Datasets

Run your prompt against all examples, check accuracy

```
correct = 0
for test in GOLDEN_DATASET:
    result = classify(test["input"])
    if result["category"] == test["expected_category"]:
        correct += 1
print(f"Accuracy: {correct}/{len(GOLDEN_DATASET)}")
```

Evaluation Strategy 3: LLM-as-Judge (Intro)

Use a **second LLM call** to evaluate the first LLM's output:

```
def judge_output(prompt, output, rubric):
    judge_messages = [
        {"role": "system", "content":
            "You are an output quality judge. Score the following "
            "LLM output on a scale of 1-5 for each criterion. "
            "Respond as JSON: {criterion: score, ...}"},
        {"role": "user", "content": f"""
Original prompt: {prompt}
LLM output: {output}
Rubric: {rubric}
"""},
    ]
    return get_completion(judge_messages, temperature=0.0)
```

Evaluation Strategy 3: LLM-as-Judge (Intro)

Pros: Scales to thousands of evaluations

Cons: The judge can be wrong too! Validate against manual scores first.

> We'll go deeper into LLM-as-judge on Day 4 (evaluation harness lab).

Versioning Prompts

Treat Prompts Like Code

KavinSchool

Prompt Versioning: Why?

Prompts are **code**. They affect behavior, quality, and cost. Version them accordingly.

```
# ❌ Bad: prompt buried in business logic, no version tracking
response = get_completion([
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": f"Summarize: {text}"},
])
```

Prompt Versioning: Why?

```
# ✅ Good: prompt templates, versioned, configurable
PROMPTS = {
    "summarize_v3": {
        "system": "You are a concise technical writer. "
                  "Summarize in exactly 3 bullet points. "
                  "Each bullet must be one sentence.",
        "user_template": "Summarize the following:\n\n---\n{text}\n---",
        "temperature": 0.3,
        "max_tokens": 200,
    },
}
```

Prompt Templates: A Simple Pattern

```
# prompts.py - your prompt configuration file
PROMPTS = {
    "code_review": {
        "version": "v2.1",
        "system": """You are a senior code reviewer.
Focus on: bugs, security, performance.
Respond as a numbered list.
Max 5 items. One sentence each.""",
        "user_template": "Review this {language} code:\n\n```{language}\n{code}\n```",
        "temperature": 0.0,
        "max_tokens": 500,
    },
}
```

Prompt Templates: A Simple Pattern

```
def run_prompt(prompt_name, **kwargs):
    config = PROMPTS[prompt_name]
    messages = [
        {"role": "system", "content": config["system"]},
        {"role": "user", "content": config["user_template"].format(**kwargs)},
    ]
    return get_completion(messages, temperature=config["temperature"],
                          max_tokens=config["max_tokens"])
```

Prompt Configuration Files (YAML/JSON)

For larger projects, store prompts in config files:

```yaml
# prompts/code_review.yaml
name: code_review
version: "2.1"
description: "Reviews code for bugs, security, and performance"
system: |
  You are a senior code reviewer.
  Focus on: bugs, security, performance.
  Respond as a numbered list.
  Max 5 items. One sentence each.
user_template: |
  Review this {language} code:

  ```{language}
  {code}
```
```

Prompt Configuration Files (YAML/JSON)

```yaml
parameters:
  temperature: 0.0
  max_tokens: 500
```

> Track prompt changes in Git. Review prompt changes in PRs just like code changes.